

HELM

Comprehensive Study Notes

CKA · CKAD · KCNA Exam Preparation

Zero to Production-Expert Level · Helm v3

[Charts](#) · [Templates](#) · [Releases](#) · [Repositories](#) · [Hooks](#)

[Security](#) · [CI/CD](#) · [Troubleshooting](#) · [95 Q&A](#)

Helm Master Guide

PART 1

PART 1: FOUNDATIONS

CHAPTER 1

What Is Helm and Why Does It Exist?

The Problem Before Helm

Managing a microservices application with 10 components quickly produces 60–80 YAML files. Five compounding problems emerge:

- **Duplication.** Image tags, limits, labels, and namespace names repeat across dozens of files.
- **No versioning.** No concept of *version 1.2.3 of my deployment* — rollback requires manually tracking old files.
- **No lifecycle management.** `kubectl apply` has no concept of which resources belong together as one application.
- **No templating.** Dev vs prod means near-identical YAML with only a few values changed.
- **No dependency management.** No built-in way to declare that your app requires a specific database chart.

The Three Core Concepts

Chart — The Package. A collection of files describing Kubernetes resources. One chart can create many different installations. Stored as `.tgz` archives.

Release — The Instance. A named installation of a chart into a cluster with a specific set of values. Installing `bitnami/mysql` twice creates two independent releases, each with its own history.

Repository — The Registry. An HTTP server serving chart packages and an `index.yaml` listing all available charts. The global public index is **Artifact Hub** (artifacthub.io).

NOTE: A **Chart** is like a class. A **Release** is like an instance. A **Repository** is like npm or PyPI.

Helm v2 vs Helm v3 — The Tiller Revolution

Aspect	Helm v2 (Tiller)	Helm v3 (no Tiller)
Architecture	CLI + Tiller pod in cluster	CLI only — no server component
Security	Tiller ran with cluster-admin	Uses user's kubeconfig RBAC directly
Release storage	ConfigMaps in kube-system	Secrets in release namespace
3-way merge	No — 2-way only	Yes — old manifest, new manifest, live state
Multi-tenancy	Difficult — global permissions	Native namespace isolation

COMMON MISTAKE: Anyone who could reach Tiller could perform *any* Kubernetes action, bypassing RBAC completely. Helm v3 fixes this by using the user's own credentials.

3-Way Strategic Merge (v3)

On helm upgrade, Helm v3 compares: the **old manifest** (stored in the release Secret), the **new manifest** (rendered from the new chart), and the **live state** (current cluster state, which may have been manually edited). This preserves manual kubectl changes. Helm v2's 2-way merge ignored live state.

When NOT to Use Helm

Scenario	Better Tool	Reason
Simple config overlays	Kustomize	YAML patching without a template language
Complex stateful apps	Operator Pattern	Custom controllers with reconciliation loops
Multi-cluster drift detection	ArgoCD / Flux	Continuous reconciliation, not one-shot deploy
Policy enforcement	Kyverno / OPA Gatekeeper	Admission webhooks, not package managers

NOTE: Helm and Kustomize are complementary. ArgoCD and Flux both support Helm natively — they run helm template internally. In production GitOps you use Helm for packaging *and* ArgoCD for delivery.

Chapter Summary

- ✓ Helm solves YAML duplication, versioning, lifecycle management, and dependency management.
- ✓ Core concepts: Chart (package), Release (instance), Repository (registry).
- ✓ Helm v3 removed Tiller — uses user's RBAC credentials directly.
- ✓ 3-way strategic merge preserves manual live-state changes during upgrades.

CHAPTER 2

Architecture Deep Dive

Helm Client Architecture (v3)

The helm binary is the only component. On each command:

- helm reads \$KUBECONFIG or ~/.kube/config.
- helm loads the chart and renders Go templates to produce plain Kubernetes YAML.
- helm communicates directly with the Kubernetes API server using your kubeconfig credentials.
- helm stores the release record as a Kubernetes Secret in the release's namespace.

Release Storage and Backends

Driver	Backend	Default?	Use Case
secret	Kubernetes Secrets (base64+gzip)	YES (v3)	Production — encrypted at rest with etcd encryption
configmap	Kubernetes ConfigMaps	No	Debugging — human-readable but unencrypted
memory	In-process memory	No	Testing — lost on process exit
sql	PostgreSQL / SQLite	No	Large clusters with many releases

Each revision is stored as `sh.helm.release.v1.<release>.v<N>`. The Secret's release field is base64-encoded, gzip-compressed JSON containing the full manifest, values, and metadata.

```
# List all Helm release secrets
kubectl get secrets -n my-ns --field-selector type=helm.sh/release.v1

# Decode a specific release secret (revision 3) — note double base64 -d
kubectl get secret sh.helm.release.v1.myapp.v3 -n my-ns \
  -o jsonpath='{.data.release}' | base64 -d | base64 -d | gunzip | python3 -m json.tool
```

NOTE: The release Secret is base64-encoded **twice** then gzip-compressed. The outer base64 is Kubernetes Secret encoding; the inner is Helm's own encoding of the JSON payload. Pipe through `base64 -d` twice.

HELM_* Environment Variables — Complete Reference

Variable	Description	Practical Use
HELM_CACHE_HOME	Chart cache directory	CI: shared cache path
HELM_CONFIG_HOME	Config directory (repos, creds)	CI: isolate creds per pipeline
HELM_DATA_HOME	Data directory (plugins)	Custom plugin storage

Variable	Description	Practical Use
HELM_DEBUG	Set '1' for verbose output	Debugging template rendering
HELM_DRIVER	Release storage backend	Use 'configmap' for debugging
HELM_KUBEAPISERVER	Override K8s API server URL	Direct access without kubeconfig
HELM_KUBECONTEXT	Override kubeconfig context	Target a specific cluster
HELM_KUBETOKEN	Bearer token for API auth	ServiceAccount token in CI/CD
HELM_KUBEINSECURE_SKIP_TLS_VERIFY	Skip TLS verify (insecure!)	Dev only — never production
HELM_NAMESPACE	Default namespace	Avoid typing -n on every command
HELM_MAX_HISTORY	Max revisions per release (default 10)	Limit Secret count in busy namespaces
HELM_PLUGINS	Override plugins directory	Custom plugin location in CI
HELM_REGISTRY_CONFIG	OCI registry auth config path	Private OCI registry credentials
HELM_REPOSITORY_CACHE	Chart repository cache path	Shared cache in CI
HELM_REPOSITORY_CONFIG	repositories.yaml path	Per-project repo config

EXAM TIP: In CI/CD set HELM_KUBETOKEN, HELM_KUBEAPISERVER, and HELM_KUBECAFILE instead of mounting a kubeconfig file — the recommended approach for GitHub Actions and GitLab CI.

Chapter Summary

- ✓ Helm v3 is client-only — no Tiller, no server-side component.
- ✓ Communicates directly with the K8s API server using your kubeconfig credentials.
- ✓ Release state stored as Kubernetes Secrets (base64+gzip encoded) by default.
- ✓ HELM_* env vars override every aspect of Helm — critical for CI/CD pipelines.

CHAPTER 3

Installation & Environment Setup

Installation Methods

Method	Command	Best For
Snap (Ubuntu/Kubuntu)	<code>sudo snap install helm --classic</code>	Easiest on Ubuntu — auto-updates
APT (Debian/Ubuntu)	<code>curl+apt-key</code> steps	System-managed, pinnable version
Homebrew (macOS)	<code>brew install helm</code>	macOS — easy, version-pinnable
Binary	Download from GitHub Releases → <code>/usr/local/bin/</code>	Exact version control, air-gapped
Script	<code>curl .../get-helm-3 bash</code>	One-liner — verify signature first

```
helm version          # verify install
helm env              # show all HELM_* variables
source <(helm completion bash) # enable tab completion
echo "source <(helm completion bash)" >> ~/.bashrc
```

Essential Plugins

Plugin	Purpose
helm-diff	Preview diff between deployed and proposed changes before every upgrade
helm-secrets	Encrypt values files with Mozilla SOPS / Vault — safe git storage
helm-unittest	Unit testing for chart templates
helm-docs	Auto-generate README documentation from chart values

```
helm plugin install https://github.com/databus23/helm-diff
helm plugin install https://github.com/jkroepke/helm-secrets
helm plugin list
helm plugin update diff
```

Chapter Summary

- ✓ `snap install helm --classic` is the simplest method on Kubuntu/Ubuntu.
- ✓ `helm version` and `helm env` verify installation and show all configuration.

✓ helm-diff (before upgrades) and helm-secrets (encrypted values) are the two most important production plugins.

PART 2

PART 2: CORE CONCEPTS IN DEPTH

CHAPTER 4

Charts: Anatomy and Structure

Chart Directory Structure

```
mychart/
├── Chart.yaml           # REQUIRED: metadata (name, version, appVersion, dependencies)
├── values.yaml          # Default configuration values
├── values.schema.json   # Optional: JSON Schema for values validation
├── charts/              # Downloaded sub-chart archives (.tgz)
├── crds/                # CRD manifests – applied FIRST before all templates
├── templates/
│   ├── NOTES.txt        # Post-install instructions (printed, not deployed to K8s)
│   ├── _helpers.tpl     # Named templates via define/include – NOT rendered as manifests
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── serviceaccount.yaml
│   ├── hpa.yaml
│   └── tests/
│       └── test-connection.yaml # helm.sh/hook: test
└── .helmignore          # Patterns excluded from helm package (like .gitignore)
```

```
helm create mychart # scaffold a new chart with all standard files
```

Chart.yaml — Complete Field Reference

```
apiVersion: v2           # REQUIRED. 'v2' for Helm v3. 'v1' is Helm v2 only.
name: mychart            # REQUIRED. Chart name – must match directory name.
version: "1.2.3"         # REQUIRED. Chart version (SemVer 2). Increment on any chart change.
appVersion: "1.25.0"     # OPTIONAL. App version – informational only, Helm ignores it.
description: Deploys nginx to Kubernetes
type: application        # 'application' (default) or 'library' (named templates only)

keywords: [nginx, web]
home: https://nginx.org
maintainers:
  - name: Safdar Ali Shah
    email: safdar@example.com

dependencies:
  - name: postgresql
    version: "12.x.x"      # SemVer constraint
    repository: "https://charts.bitnami.com/bitnami"
    condition: postgresql.enabled # enable/disable via values
    alias: db              # rename sub-chart in values.yaml
```

EXAM TIP: version vs appVersion: version is the chart package version — increment whenever you change templates or values structure. appVersion is purely informational; Helm never uses it for logic. The actual image tag is controlled by values.yaml.

_helpers.tpl — Named Templates

Files beginning with `_` are **never** rendered as Kubernetes manifests. They define reusable partials shared across all templates via `include`:

```
{{/* Full qualified app name */}}
{{- define "mychart.fullname" -}}
{{- if .Values.fullnameOverride }}
{{- .Values.fullnameOverride | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- printf "%s-%s" .Release.Name (default .Chart.Name .Values.nameOverride) | trunc 63 | trimSuffix "-" }}
{{- end }}
{{- end }}

{{/* Common labels – applied to every resource */}}
{{- define "mychart.labels" -}}
helm.sh/chart: {{ printf "%s-%s" .Chart.Name .Chart.Version | replace "+" "_" | trunc 63 }}
app.kubernetes.io/name: {{ default .Chart.Name .Values.nameOverride | trunc 63 }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}

{{/* Selector labels – IMMUTABLE after first deploy, never include version */}}
{{- define "mychart.selectorLabels" -}}
app.kubernetes.io/name: {{ default .Chart.Name .Values.nameOverride | trunc 63 }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}
```

COMMON MISTAKE: Selector labels in `matchLabels` are **immutable** after a Deployment is created. Never put `helm.sh/chart` or `version` in selectors — they change each upgrade, making the Deployment impossible to update.

Chapter Summary

- ✓ version (chart SemVer) vs appVersion (informational only) — critical distinction.
- ✓ type: library charts hold only named templates — no deployable resources.
- ✓ Files starting with `_` are not rendered — they hold reusable define blocks.
- ✓ NOTES.txt is rendered post-install and printed to stdout — supports Go templates.
- ✓ Selector labels are immutable — only app name and release instance, never version.

CHAPTER 5

Go Templating in Helm

Whitespace Control

Helm uses Go's text/template package with the Sprig function library. YAML is whitespace-sensitive, making whitespace control critical:

```
{{ .Values.x }}      # preserve surrounding whitespace
{{- .Values.x }}     # strip whitespace/newlines BEFORE this action
{{ .Values.x -}}     # strip whitespace/newlines AFTER this action
{{- .Values.x -}}    # strip both sides
```

Built-in Objects

Object.Field	Type	Description
.Release.Name	string	The release name (e.g. my-nginx)
.Release.Namespace	string	Target namespace
.Release.IsInstall	bool	True only on initial helm install
.Release.IsUpgrade	bool	True on upgrade or rollback
.Release.Revision	int	Revision number — 1 on install, increments each operation
.Release.Service	string	Always 'Helm'
.Chart.Name	string	Chart name from Chart.yaml
.Chart.Version	string	Chart version (SemVer 2)
.Chart.AppVersion	string	App version — informational only
.Values.*	any	Merged values — user overrides + chart defaults
.Capabilities.KubeVersion.GitVersion	string	Full K8s version string e.g. v1.28.3
.Capabilities.APIVersions.Has 'a/v/K'	bool	Check API group/version/kind availability
.Files.Get 'path'	string	Read a non-template file from the chart
.Template.Name	string	Current template file path

Essential Sprig Functions

Function	Example
default	{{ .Values.tag default "latest" }}
required	{{ required "tag required" .Values.tag }}

Function	Example
quote	<code>{{ .Values.name quote }} → "my-name"</code>
upper/lower	<code>{{ .Values.env upper }} → PRODUCTION</code>
replace	<code>{{ .Chart.Version replace "+" "_" }}</code>
toYaml	<code>{{ .Values.resources toYaml nindent 2 }}</code>
include	<code>{{ include "chart.labels" . nindent 4 }}</code>
tpl	<code>{{ tpl .Values.configTemplate . }}</code>
b64enc	<code>{{ .Values.password b64enc }}</code>
sha256sum	<code>{{ .Values.config toYaml sha256sum }}</code>
randAlphaNum	<code>{{ randAlphaNum 32 }} → random 32-char string</code>
semverCompare	<code>{{ if semverCompare ">=1.19" .Capabilities.KubeVersion.GitVersion }}</code>

include vs template — Critical Distinction

include returns the rendered text as a **string** that can be piped. template writes directly to output and **cannot be piped**. Always use include:

```
# CORRECT – include is pipe-able:
metadata:
  labels:
    {{- include "mychart.labels" . | nindent 4 }}

# WRONG – template cannot be piped to nindent – indentation breaks YAML:
metadata:
  labels:
    {{- template "mychart.labels" . }}
```

Control Structures

if / else / else if

```

{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
{{- end }}

{{- if eq .Values.service.type "LoadBalancer" }}
  type: LoadBalancer
{{- else if eq .Values.service.type "NodePort" }}
  type: NodePort
{{- else }}
  type: ClusterIP
{{- end }}

{{- if and .Values.ingress.enabled .Values.ingress.tls }} ... {{- end }}
{{- if or .Values.metrics.enabled .Values.tracing.enabled }} ... {{- end }}

```

range — Iteration

```

# Iterate a list
env:
{{- range .Values.extraEnvVars }}
  - name: {{ .name }}
    value: {{ .value | quote }}
{{- end }}

# Iterate a map — $key, $value
{{- range $key, $value := .Values.labels }}
  {{ $key }}: {{ $value | quote }}
{{- end }}

```

with — Scope Narrowing

```

{{- with .Values.resources }}
resources:
  {{- toYaml . | nindent 2 }}    # '.' is now .Values.resources
{{- end }}

# Use $ to access root scope inside with/range
{{- with .Values.podAnnotations }}
annotations:
  {{- toYaml . | nindent 4 }}
  managed-by: {{ $.Release.Name }}    # $ = root context always
{{- end }}

```

NOTE: Inside range and with blocks, `.` changes scope. Use `$` to access the root context (e.g. `$.Release.Name`, `$.Values.global.*`).

Chapter Summary

- ✓ Always use include over template — include returns a string that can be piped.
- ✓ Use nindent N (newline + N spaces) when embedding multi-line values in YAML.

- ✓ `$` always refers to the root context inside `range/with` blocks.
- ✓ `tpl` evaluates values as templates — powerful but dangerous with untrusted input.
- ✓ Falsy in Go templates: `false`, `0`, `nil`, empty string, empty list, empty map.

CHAPTER 6

Values: Configuration Management

Values Precedence Hierarchy (Lowest → Highest)

Priority	Source	How Applied
1 (lowest)	Chart's values.yaml	Bundled defaults — always present
2	Sub-chart values.yaml	Sub-chart's own defaults
3	Parent chart values.yaml	Parent overrides sub-chart defaults
4	-f / --values file(s)	Multiple -f: applied left-to-right, later wins
5 (highest)	--set flags	Command-line — always override everything

```
# values.yaml: replicaCount: 1 | prod.yaml: replicaCount: 3 | --set: replicaCount=5
helm install myapp ./chart -f prod.yaml --set replicaCount=5
# Result: replicaCount = 5 (--set wins)
```

--set vs -f vs Variants

Flag	Use For	Note
--set	Simple scalar overrides	Shell-expanded; avoid special chars
-f / --values	Complex structures or many overrides	Recommended for environment config
--set-string	Force string type	Prevents int/bool coercion bugs e.g. port: '8080'
--set-file	Load value from a file	Reads entire file content as string value
--set-json	Pass JSON value directly	Useful for nested objects and arrays

Global Values — Cross-Sub-Chart Communication

A special global key is accessible to **all** sub-charts as `.Values.global.*`:

```
# Parent values.yaml
global:
  imageRegistry: "harbor.example.com"
  imagePullSecrets: [{name: registry-secret}]

# In any sub-chart template:
image: {{ .Values.global.imageRegistry }}/{{ .Values.image.repository }}:{{ .Values.image.tag }}
```

values.schema.json — Input Validation

Helm validates all supplied values against this JSON Schema before rendering templates. Catches misconfiguration before any resource is deployed:

```
{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["image"],
  "properties": {
    "replicaCount": { "type": "integer", "minimum": 1, "maximum": 50 },
    "image": {
      "required": ["repository", "tag"],
      "properties": {
        "repository": { "type": "string" },
        "tag": { "type": "string" },
        "pullPolicy": { "enum": ["Always", "IfNotPresent", "Never"] }
      }
    },
    "service": {
      "properties": {
        "type": { "enum": ["ClusterIP", "NodePort", "LoadBalancer", "ExternalName"] },
        "port": { "type": "integer", "minimum": 1, "maximum": 65535 }
      }
    }
  }
}
```

COMMON MISTAKE: Never pass secrets via `--set password=secret`. The value is stored in plain base64 in the release Secret, appears in shell history, CI/CD logs, and process listings. Use Sealed Secrets, SOPS+helm-secrets, or External Secrets Operator.

Chapter Summary

- ✓ Values precedence (low→high): values.yaml → sub-chart → parent → -f files → --set.
- ✓ --set-string forces string type to prevent YAML integer/boolean coercion bugs.
- ✓ Deep merge preserves unspecified nested fields; arrays are fully replaced.
- ✓ global values (.Values.global.*) are visible to all sub-charts.
- ✓ values.schema.json validates user input before template rendering — always include it.

PART 3

PART 3: HELM CLI — COMPLETE REFERENCE

CHAPTER 7

Repository Management

Core Repository Commands

```
helm repo add bitnami      https://charts.bitnami.com/bitnami
helm repo add jetstack     https://charts.jetstack.io
helm repo list              # list configured repos
helm repo update           # fetch latest index.yaml from all repos
helm repo remove bitnami

helm search repo nginx     # search local cache (must repo update first)
helm search repo nginx --versions # all available versions
helm search hub wordpress  # search Artifact Hub (internet, no local config needed)

helm show chart bitnami/nginx # Chart.yaml
helm show values bitnami/nginx # default values.yaml
helm show all bitnami/nginx   # everything
```

OCI Registries (Helm 3.8+)

Charts can be stored in OCI registries alongside container images. No helm repo add needed:

```
helm registry login registry-1.docker.io --username myuser
helm pull oci://registry-1.docker.io/bitnamicharts/nginx --version 15.1.0
helm install my-nginx oci://registry-1.docker.io/bitnamicharts/nginx
helm push mychart-1.0.0.tgz oci://my-registry.example.com/charts
```

Chapter Summary

- ✓ helm repo add/list/update/remove manages local repository configuration.
- ✓ helm search repo searches local cache; helm search hub searches Artifact Hub globally.
- ✓ OCI registries (Helm 3.8+) store charts alongside images — no index.yaml needed.

CHAPTER 8

Installing and Managing Releases

helm install — Complete Syntax

```
# Install forms
helm install <RELEASE> <CHART> [flags]
helm install my-nginx bitnami/nginx
helm install my-nginx ./nginx-chart           # local directory
helm install my-nginx ./nginx-1.0.0.tgz      # local .tgz archive
helm install my-nginx oci://registry/charts/nginx

# Most important flags
helm install my-nginx bitnami/nginx \
  --namespace production \      # target namespace
  --create-namespace \          # create ns if it doesn't exist
  -f prod-values.yaml \         # custom values file
  --set image.tag=1.25.3 \      # inline override (highest priority)
  --version 15.1.0 \           # specific chart version
  --atomic \                    # wait for Ready + auto-rollback on failure
  --timeout 10m \              # wait timeout (default 5m)
  --dry-run \                   # simulate – hits API server, no deploy
  --debug \                     # verbose – prints rendered manifests
  --generate-name \             # auto-generate release name
  --dependency-update           # update chart deps before install
```

--dry-run vs helm template

Aspect	--dry-run	helm template
Connects to cluster?	YES — validates against live API	NO — purely local
.Capabilities populated?	YES — real cluster capabilities	NO — static defaults
lookup() works?	YES	NO — returns empty
Works without cluster?	NO	YES — ideal for offline CI
Best for	Final pre-deploy validation	Template debugging and CI linting

EXAM TIP: Always use `--atomic` for production installs. Without it, a failed install leaves the release in *pending-install* state that blocks subsequent installs with the same name.

helm list and Status Values

```
helm list -n production    # releases in a namespace
helm list -A              # all releases in ALL namespaces
helm list --all           # include failed/pending releases

# STATUS values in helm list:
# deployed                - currently installed and healthy
# failed                  - last operation failed
# pending-install / pending-upgrade / pending-rollback - stuck or in progress
# superseded              - replaced by a newer revision
# uninstalled             - removed with --keep-history
```

helm status and helm get Subcommands

```
helm status my-nginx      # summary + NOTES.txt
helm get all my-nginx     # everything combined
helm get values my-nginx  # user-supplied values only
helm get values my-nginx --all # ALL values (defaults + user)
helm get manifest my-nginx # rendered K8s manifests as deployed
helm get hooks my-nginx   # hook manifests
helm get notes my-nginx   # NOTES.txt output
```

Chapter Summary

- ✓ helm install --atomic = --wait + automatic rollback on failure — use in production.
- ✓ --dry-run validates against the live API; helm template is fully offline.
- ✓ helm list STATUS: deployed, failed, pending-install/upgrade/rollback, superseded.
- ✓ helm get manifest shows what was actually deployed; helm get values shows config.

CHAPTER 9

Upgrading, Rolling Back, and History

helm upgrade — Key Flags

```
helm upgrade my-nginx bitnami/nginx \
  -f prod-values.yaml           \
  --version 15.2.0              \  # upgrade to specific chart version
  --install                     \  # upgrade-or-install (idempotent pattern)
  --atomic                      \  # wait + rollback on failure
  --cleanup-on-fail             \  # delete newly created resources if upgrade fails
  --reset-values                \  # discard previous values; start fresh
  --reuse-values                 \  # keep previous values (avoid in automation)
  --force                       \  # delete + recreate resources (brief downtime)
```

--reset-values vs --reuse-values

Flag	Behavior	Risk
--reset-values	Discard ALL previous values; start fresh from defaults + new flags	LOW — deterministic
--reuse-values	Merge previous release values; add new flags on top	HIGH — drift accumulates invisibly
(neither)	Re-merge chart defaults for new keys; keep user values	MEDIUM — standard for full -f files

COMMON MISTAKE: Avoid --reuse-values in automated pipelines. Values from months ago persist invisibly, causing configuration drift. Best practice: supply a complete -f values.yaml and use --reset-values.

helm history and helm rollback

```
helm history my-nginx -n production
# REVISION  UPDATED          STATUS      CHART          DESCRIPTION
# 1          2024-01-10       superseded  nginx-14.0.0   Install complete
# 2          2024-01-12       superseded  nginx-15.0.0   Upgrade complete
# 3          2024-01-15       deployed   nginx-15.1.0   Upgrade complete

helm rollback my-nginx 2 -n production  # rollback to specific revision
helm rollback my-nginx -n production    # rollback to previous revision
helm rollback my-nginx 0 -n production   # rollback to last DEPLOYED (skips failed)
```

NOTE: helm rollback creates a **new revision** by re-running the old manifest — it does NOT revert in-place. Database schema migrations are **not** reverted; only K8s object definitions are re-applied.

Fixing Stuck Releases

```
# Symptom: 'UPGRADE FAILED: another operation is in progress'
helm history my-nginx -n production          # shows pending-upgrade status

# Fix: rollback to last deployed revision
helm rollback my-nginx 0 -n production

# Nuclear: delete all release secrets and reinstall
kubectl delete secrets -n production -l owner=helm,name=my-nginx
helm install my-nginx bitnami/nginx -n production
```

Chapter Summary

- ✓ `helm upgrade --atomic` is the production-safe upgrade pattern (wait + auto-rollback).
- ✓ Avoid `--reuse-values` in automation — causes invisible configuration drift.
- ✓ `helm rollback` creates a NEW revision — does NOT revert Kubernetes objects in-place.
- ✓ Rollback to revision 0 = last successfully deployed revision.

CHAPTER 10

Chart Dependencies

Declaring and Downloading Dependencies

```
# Chart.yaml – declare dependencies
dependencies:
  - name: postgresql
    version: "12.x.x"                # SemVer constraint
    repository: "https://charts.bitnami.com/bitnami"
    condition: postgresql.enabled    # enable/disable via values
    alias: db                        # rename in values.yaml

  - name: redis
    version: "17.x.x"
    repository: "https://charts.bitnami.com/bitnami"
    condition: redis.enabled
    tags: [cache]                    # group enable/disable

# Download dependencies to charts/
helm dependency update  # resolves versions, downloads, generates Chart.lock
helm dependency build   # downloads from Chart.lock exactly (CI/deterministic)
helm dependency list    # show resolved dependencies
```

Chart.yaml vs Chart.lock

File	Purpose	Commit to Git?
Chart.yaml (dependencies)	Declares version constraints	YES — always
Chart.lock	Pins exact resolved versions + checksums	YES — ensures reproducibility
charts/ directory	Downloaded sub-chart archives	Optional — regenerate with dep build

Passing Values to Sub-Charts

```
# Parent values.yaml – use sub-chart name (or alias) as top-level key
postgresql:
  enabled: true
  auth:
    postgresPassword: "secret"
    database: "myapp"

db:          # alias 'db' from Chart.yaml
  auth:
    password: "anothersecret"

# Global values – accessible to ALL sub-charts via .Values.global.*
global:
  imageRegistry: "harbor.example.com"
  storageClass: "fast-ssd"
```

Chapter Summary

- ✓ Chart.lock pins exact versions — both Chart.yaml and Chart.lock should be committed.
- ✓ helm dep update re-resolves; helm dep build uses Chart.lock (deterministic, CI-safe).
- ✓ Pass values to sub-charts using the sub-chart name (or alias) as key in parent values.yaml.
- ✓ global values (.Values.global.*) are accessible to every sub-chart.

CHAPTER 11

Testing, Linting, and Packaging

Lint, Template, and Test

```
# Lint – static analysis of chart structure and template syntax
helm lint ./mychart
helm lint ./mychart --strict           # warnings become errors
helm lint ./mychart -f prod-values.yaml

# Render templates locally (no cluster needed)
helm template my-release ./mychart     # all templates
helm template my-release ./mychart -s templates/deployment.yaml
helm template my-release ./mychart -f values.yaml > rendered.yaml

# Schema-validate rendered YAML against K8s API schemas
helm template my-release ./mychart | kubeconform -kubernetes-version 1.28

# Run test pods inside the cluster
helm test my-release -n production
helm test my-release --logs           # show test pod logs
```

CI/CD Pipeline Order

Step	Command	Purpose
1. Lint	helm lint ./chart --strict	Syntax and structure validation
2. Render	helm template rel ./chart -f vals.yaml > m.yaml	Produce manifest YAML
3. Schema	kubeconform < m.yaml	Validate against K8s API schemas
4. Policy	confest test m.yaml	Enforce OPA policies
5. Diff	helm diff upgrade rel ./chart -f vals.yaml	Preview changes
6. Deploy	helm upgrade --install rel ./chart --atomic	Deploy to cluster
7. Smoke	helm test rel -n ns	Post-deploy test pods

Packaging and Distribution

```
helm package ./mychart # creates mychart-1.0.0.tgz
helm package ./mychart --destination ./releases/

# Sign with GPG
helm package ./mychart --sign --key "My Key" --keyring ~/.gnupg/secring.gpg
# Creates: mychart-1.0.0.tgz + mychart-1.0.0.tgz.prov

helm verify mychart-1.0.0.tgz # verify signature
helm install myapp mychart-1.0.0.tgz --verify # install only if verified

helm repo index ./releases/ --url https://charts.example.com # generate index.yaml
helm push mychart-1.0.0.tgz oci://registry.example.com/charts
```

Chapter Summary

- ✓ helm lint for syntax; helm template for local rendering; helm test for cluster integration tests.
- ✓ Pipe helm template to kubeconform for full K8s schema validation without a cluster.
- ✓ GPG signing creates a .prov provenance file for supply chain verification.
- ✓ helm repo index generates the index.yaml needed to serve an HTTP chart repository.

PART 4

PART 4: ADVANCED HELM

CHAPTER 12

Helm Hooks

Hook Types and Lifecycle

Hooks are Kubernetes resources (Jobs, Pods) annotated with `helm.sh/hook` that Helm executes at specific release lifecycle points, waiting for completion before proceeding.

Hook Type	Runs When	Typical Use Case
pre-install	Before chart resources created	Database init, certificate generation
post-install	After all resources Ready	Slack notification, smoke test
pre-upgrade	Before upgrade applied	Database migration, data backup
post-upgrade	After upgrade complete	Cache warm-up, smoke test
pre-rollback	Before rollback begins	Backup current state
post-rollback	After rollback complete	Notification, verification
pre-delete	Before uninstall begins	Data backup, graceful drain
post-delete	After all resources deleted	Clean up external resources
test	Only when helm test explicitly run	Integration / smoke tests

Hook Annotations — Full Example

```
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ include "mychart.fullname" . }}-db-migrate
  annotations:
    "helm.sh/hook": pre-upgrade           # hook type(s) – comma-separate for multiple
    "helm.sh/hook-weight": "-5"          # execution order: lower = earlier (must be string)
    "helm.sh/hook-delete-policy": before-hook-creation, hook-succeeded
spec:
  template:
    spec:
      restartPolicy: Never                # Jobs must have Never or OnFailure
      containers:
        - name: migrate
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          command: ["python", "manage.py", "migrate"]
```

Hook Weights and Delete Policies

`hook-weight` controls execution order when multiple hooks of the same type exist. Lower (more negative) weights run first. The value **must be a string**.

Delete Policy	When Resource Is Deleted
before-hook-creation (default)	Before a new hook of the same name is created — prevents accumulation of old pods
hook-succeeded	After the hook completes successfully — keeps failed resources for debugging
hook-failed	After the hook fails — automatically cleans up failed jobs

COMMON MISTAKE: If a hook Job fails, Helm marks the entire release as *failed* and stops processing. NOT automatically rolled back unless `--atomic` was used. Delete the failed Job manually, fix the issue, then retry the helm operation.

Chapter Summary

- ✓ Hooks are K8s resources with `helm.sh/hook` annotation — run at lifecycle points.
- ✓ `hook-weight` controls order: lower = earlier. Value must be a string ('-5', not -5).
- ✓ Standard delete policy: `before-hook-creation, hook-succeeded`.
- ✓ A failed hook marks the release failed — use `--atomic` to auto-rollback.

CHAPTER 13

Helm Security

What a Release Secret Contains

Every Helm release revision is stored as a Kubernetes Secret. The decoded JSON contains: release name, namespace, status, **config** (user-supplied values — including any secrets passed via `--set`), and **manifest** (full rendered YAML). Only base64-encoded — **not** encrypted.

```
kubectl get secrets -n production --field-selector type=helm.sh/release.v1

# Full decode — note double base64 -d
kubectl get secret sh.helm.release.v1.myapp.v3 -n production \
  -o jsonpath='{.data.release}' | base64 -d | base64 -d | gunzip | python3 -m json.tool
```

COMMON MISTAKE: Passing `--set password=secret` stores that password in plain base64 in the release Secret — readable by anyone with `get secrets` permission, visible in shell history, CI/CD logs, and process listings.

Secrets Management Patterns

Pattern	Tool	How It Works
Sealed Secrets	Bitnami Sealed Secrets	Encrypt with cluster public key; safe to commit to git
SOPS + helm-secrets	Mozilla SOPS + helm-secrets	Encrypt values files; decrypt at deploy time
External Secrets Operator	External Secrets Operator	Sync from AWS SM, Vault, GCP SM into K8s Secrets
Vault Agent Injector	HashiCorp Vault	Inject secrets as files/env vars via sidecar
CI/CD pre-created Secrets	GitHub Actions, GitLab CI	Create K8s Secret before helm install; reference by name

Minimum RBAC for helm install

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: helm-deployer
  namespace: production
rules:
  - apiGroups: [""]
    resources: ["secrets"] # Helm release storage – required
    verbs: ["get", "list", "create", "update", "delete"]
  - apiGroups: ["apps"]
    resources: ["deployments", "replicasets"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
  - apiGroups: [""]
    resources: ["services", "configmaps", "serviceaccounts"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
  - apiGroups: ["networking.k8s.io"]
    resources: ["ingresses"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
```

Chapter Summary

- ✓ Release secrets contain full manifests and values — only base64, not encrypted.
- ✓ Never use --set for passwords — use Sealed Secrets, SOPS, or External Secrets Operator.
- ✓ Minimum RBAC: Secrets CRUD + permissions for all resource types the chart creates.
- ✓ GPG chart signing creates .prov provenance files for supply chain verification.

CHAPTER 14

Helm in CI/CD and Production

Production Deploy Pipeline

```
# 1. Lint
helm lint ./chart --strict -f values/base.yaml

# 2. Render
helm template $RELEASE ./chart -f values/base.yaml -f values/$ENV.yaml > manifest.yaml

# 3. Schema validate
kubecnf -kubernetes-version 1.28 manifest.yaml

# 4. Policy check
conftest test manifest.yaml --policy ./opa-policies/

# 5. Diff (upgrade only – shows exact changes before apply)
helm diff upgrade $RELEASE ./chart -f values/base.yaml -f values/$ENV.yaml --allow-unreleased

# 6. Deploy
helm upgrade --install $RELEASE ./chart \
  --namespace $NAMESPACE --create-namespace \
  -f values/base.yaml -f values/$ENV.yaml \
  --atomic --timeout 10m \
  --description "CI deploy: $COMMIT_SHA"

# 7. Smoke test
helm test $RELEASE --namespace $NAMESPACE --timeout 5m
```

ArgoCD and Flux with Helm

ArgoCD does NOT run helm install. It runs helm template and applies with kubectl apply.

Consequences:

- Helm hooks do **not** run in ArgoCD (hooks are lifecycle annotations; ArgoCD uses kubectl apply).
- No helm history — ArgoCD uses its own sync history.
- NOTES.txt is not shown.

Flux HelmRelease CRD uses the Helm SDK directly — hooks work and history is maintained.

Helmfile — Declarative Multi-Release Management

```
# helmfile.yaml
repositories:
  - name: bitnami
    url: https://charts.bitnami.com/bitnami
environments:
  production: { values: [env/production.yaml] }
  staging:    { values: [env/staging.yaml]   }
releases:
```

```

- name: my-app
  namespace: production
  chart: ./charts/my-app
  values:
    - values/base.yaml
    - values/{{ .Environment.Name }}.yaml
  set:
    - name: image.tag
      value: {{ requiredEnv "IMAGE_TAG" }}
- name: postgresql
  namespace: production
  chart: bitnami/postgresql
  version: "12.x.x"
  needs: [production/my-app]      # dependency ordering
# helmfile apply                  - install/upgrade all releases
# helmfile -e production apply   - specific environment
# helmfile diff                  - diff all releases

```

Chart Best Practices

Practice	Rule
Naming	Lowercase and hyphens only — no underscores or dots
Image tags	Never use 'latest' as default — pin to specific tag or SHA digest
Resource naming	Always use {{ include 'chart.fullname' . }} — prevents namespace collisions
Standard labels	Always: helm.sh/chart, app.kubernetes.io/name, instance, version, managed-by
Resource limits	Always define both requests AND limits
Health probes	Always include liveness and readiness probes
ServiceAccount	Always create a dedicated SA — principle of least privilege
NOTES.txt	Always include — document post-install access instructions
Schema	Add values.schema.json to catch misconfiguration before any resource is deployed

Chapter Summary

- ✓ CI/CD: lint → template → kubeconform → conftest → diff → upgrade --atomic → test.
- ✓ ArgoCD uses helm template; Flux HelmRelease uses native Helm SDK (hooks work).
- ✓ Helmfile manages multiple releases declaratively across environments.
- ✓ Selector labels are immutable — never version or chart label in matchLabels.

PART 5

PART 5: REAL-WORLD WORKFLOW

CHAPTER 15

The nginx Deployment Workflow

Step 1 — Generate Manifests with kubectl

```
# Generate deployment YAML without deploying
kubectl create deploy nginx --image=nginx:alpine --port=80 --dry-run=client -o yaml > deploy.yaml

# Generate service YAML without deploying
kubectl expose deploy nginx --target-port=80 --dry-run=client -o yaml > service.yaml
```

Step 2 — Create Chart Scaffold and Convert Templates

```
helm create nginx-deploy-chart

# templates/deployment.yaml – converted from raw YAML
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "nginx-deploy-chart.fullname" . }}
  namespace: {{ .Release.Namespace }}
  labels:
    {{- include "nginx-deploy-chart.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "nginx-deploy-chart.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "nginx-deploy-chart.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - name: http
              containerPort: {{ .Values.service.port }}
          livenessProbe:
            httpGet: { path: /, port: http }
          readinessProbe:
            httpGet: { path: /, port: http }
          {{- with .Values.resources }}
          resources:
            {{- toYaml . | nindent 12 }}
          {{- end }}
```

Step 3 — values.yaml

```

replicaCount: 1
image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: "alpine"
serviceAccount:
  create: true
  annotations: {}
  name: ""
service:
  type: ClusterIP
  port: 80
resources:
  limits:   { cpu: 100m, memory: 128Mi }
  requests: { cpu: 50m,  memory: 64Mi  }
```

Step 4 — Install, Upgrade, Rollback, Package

```

helm install  nginx-release ./nginx-deploy-chart -n default
helm upgrade  nginx-release ./nginx-deploy-chart --set replicaCount=3 -n default
helm history  nginx-release -n default
helm rollback nginx-release -n default          # rollback to previous revision
helm package  ./nginx-deploy-chart             # → nginx-deploy-chart-0.1.0.tgz
```

Troubleshooting Quick Reference

Error / Symptom	Cause	Fix
another operation in progress	Previous op left pending state	helm rollback 0
release has no deployed releases	All revisions failed	helm uninstall + reinstall
Hook job blocking release	pre/post hook Job failed	kubectrl delete job; retry
nil pointer evaluating ...tag	Parent key nil in values	Add key; use 'with' guard
cannot re-use a name in use	Release name already exists	helm upgrade --install
INSTALLATION FAILED: timed out	--atomic timeout exceeded	Increase --timeout; debug pods
helm list shows nothing	Wrong namespace	helm list -A

PART 6

**PART 6: EXAM PREPARATION — 95
Q&A;**

CHAPTER 16

Section A: Conceptual Questions (Q1–Q15)

Q1. Why was Helm created? What problem does it solve that kubectl apply -f does not?

kubectl apply deploys resources but provides no lifecycle management: no versioning, no rollback, no templating, no release naming, and no dependency management. Helm adds all of these — it packages manifests into Charts, creates named Releases with revision history, enables parameterised templates for multi-environment deployment, and manages inter-chart dependencies. kubectl apply is like copying files; Helm is a proper package manager.

Q2. Explain the difference between a Chart, a Release, and a Revision.

A CHART is the package — directory/archive of templates, values, and metadata. A RELEASE is a named instance of a chart in a specific namespace with its own history. Installing bitnami/mysql twice creates two independent releases. A REVISION is a numbered snapshot: every install creates revision 1, every upgrade or rollback creates a new revision.

Q3. Why was Tiller removed in Helm v3?

Tiller ran inside the cluster with cluster-admin privileges by default. Anyone who could reach Tiller could perform any Kubernetes action, completely bypassing RBAC. Helm v3 removed Tiller — the CLI communicates directly with the K8s API server using the user's own kubeconfig credentials, so Kubernetes RBAC applies natively.

Q4. How is Helm different from Kustomize?

Helm uses Go templates — full programming language with conditionals, loops, and functions. Kustomize uses YAML patching with no templating. Choose Helm when you need parameterisation, dependency management, or release lifecycle management. Choose Kustomize for simple overlays without logic. They are complementary — Kustomize can invoke helm template as a generator.

Q5. Why does Helm store release state as Kubernetes Secrets by default?

Secrets are stored in etcd. If etcd is encrypted at rest, Helm release data is encrypted automatically. Secrets also have a type field (helm.sh/release.v1) for easy filtering. The tradeoff: anyone with 'get secrets' permission in a namespace can read all Helm release metadata including values and rendered manifests.

Q6. What is the difference between appVersion and version in Chart.yaml?

version is the CHART package version (SemVer 2) — increment it when you change templates or values structure. appVersion is informational only — it documents what application version the chart was designed for. Helm never uses appVersion for any logic. The actual image tag is controlled by values.yaml.

Q7. Why should you never use 'latest' as an image tag in values.yaml?

'latest' creates: (1) Non-determinism — two installs at different times pull different versions. (2) Broken rollback — helm rollback re-deploys the old manifest, but 'latest' now points to a different image. (3) CI/CD unreliability — deployment state cannot be reproduced. (4) imagePullPolicy IfNotPresent may serve a cached old 'latest' image. Always pin to a specific tag or SHA digest.

Q8. What is the difference between helm template and helm install --dry-run?

helm template renders templates to YAML locally with NO cluster connection. .Capabilities uses static defaults. lookup() returns empty. Best for offline CI linting.

helm install --dry-run connects to the API server, validates against the live OpenAPI schema, populates .Capabilities from the real cluster, and makes lookup() work. Creates no resources. Best for final pre-deploy validation.

Q9. What problem do Helm hooks solve? Give three real-world examples.

Hooks solve the need to perform actions at specific lifecycle points that must complete before or after main resources are deployed.

1. DATABASE MIGRATION (pre-upgrade): Run manage.py migrate before deploying the new app version.
2. SMOKE TEST (post-install): Run a test Pod that curls the service and verifies a 200 response.
3. DATA BACKUP (pre-delete): Run a Job that dumps the database to S3 before uninstalling.

Q10. Why is --reuse-values potentially dangerous in automated pipelines?

--reuse-values takes all previous revision values and merges them with new flags. Problems: (1) Drift — an operator sets --set debug=true in an emergency; all future upgrades silently reuse debug=true. (2) Invisible accumulated values easy to miss. (3) New chart default keys may be blocked by old merged values. Best practice: supply a complete -f values.yaml and use --reset-values.

Q11. Explain the values precedence hierarchy from lowest to highest.

- 1 (LOWEST): Chart's values.yaml — bundled defaults.
- 2: Sub-chart values.yaml defaults.
- 3: Parent chart values.yaml overrides for sub-chart keys.
- 4: User -f values files — multiple -f applied left-to-right, later files win.
- 5 (HIGHEST): --set flags — always win.

Q12. Why is helm rollback not always safe for stateful applications?

helm rollback re-deploys previous revision manifests. Problems: (1) Database schema migrations from the upgrade are NOT reverted — old code may be incompatible with the new schema. (2) PVC data is NOT reverted. (3) External resources (DNS, cloud resources) may not be cleaned up. (4) Immutable K8s fields (selector labels, StorageClass on PVCs) may cause rollback to fail. Rollback is safe for stateless apps only.

Q13. What is the purpose of _helpers.tpl?

_helpers.tpl stores named template definitions ({{- define ... -}}) reused across all templates. Benefits: DRY principle, consistency (all resources get identical labels from one source), and the underscore prefix ensures it is never rendered as a K8s manifest. Standard helm create scaffold always includes _helpers.tpl.

Q14. What is the difference between helm dependency update and helm dependency build?

helm dependency update: reads Chart.yaml constraints, resolves latest matching versions, downloads them, regenerates Chart.lock. Requires internet access.

helm dependency build: reads Chart.lock exactly, downloads those pinned versions. Does NOT re-resolve. Use in CI/CD for deterministic builds.

Q15. What is the 3-way strategic merge in Helm v3?

On upgrade, Helm v3 compares three states: (1) old manifest stored in the release Secret, (2) new rendered manifest, and (3) current live state in the cluster. This lets Helm detect and preserve manual kubectl changes. Helm v2's 2-way merge only compared old and new manifests, ignoring live state entirely.

CHAPTER 17

Section B: Architecture & CLI Questions (Q16–Q55)

Q16. How does Helm v3 communicate with the Kubernetes API server?

Helm uses the same client-go libraries as kubectl. It reads \$KUBECONFIG, renders templates to YAML, then sends REST calls directly to the K8s API server. Helm uses the user's own credentials — Kubernetes RBAC evaluates requests natively. Release records are stored as Secrets via the Secrets API. No proxy, no intermediate component.

Q17. How do you read a Helm release secret with kubectl?

```
kubectl get secrets -n --field-selector type=helm.sh/release.v1
```

Decode: `kubectl get secret sh.helm.release.v1..v -n \ -o jsonpath='{.data.release}' | base64 -d | base64 -d | gunzip | python3 -m json.tool`

Double base64 -d: the outer is the K8s Secret data encoding; the inner is Helm's own encoding.

Q18. What does a decoded Helm release secret contain?

Decoded JSON contains: name (release name), namespace, info (status, first/last deployed, description), chart (Chart.yaml + templates), config (user-supplied values — including secrets passed via --set!), manifest (full rendered YAML), hooks, and version (revision number).

Q19. Explain every column in the output of helm list.

NAME: release name. NAMESPACE: target namespace. REVISION: current revision number. UPDATED: timestamp of last operation. STATUS: deployed/failed/pending-*/superseded. CHART: chart name-version (e.g. nginx-15.1.0). APP VERSION: appVersion from Chart.yaml — informational only.

Q20. What does helm get values vs helm get values --all show?

helm get values: shows ONLY user-supplied values (via -f or --set). Not chart defaults.

helm get values --all: shows COMPLETE merged values — chart defaults plus all user overrides. This is the full set used to render the current revision's templates.

Q21. What is the difference between helm status and helm get all?

helm status: summary — release name, deployed time, namespace, status, and NOTES.txt. Quick health check.

helm get all: full detail — values, rendered manifest, hooks, NOTES.txt. Use for debugging.

Q22. How do you install a chart with a custom values file AND one additional override?

```
helm install myapp ./chart -f custom-values.yaml --set image.tag=v2.0.0
```

--set has higher priority than -f, so image.tag=v2.0.0 overrides whatever is in custom-values.yaml.

Q23. How do you upgrade a release and ensure it rolls back automatically on failure?

```
helm upgrade myapp ./chart --atomic --timeout 10m
```

--atomic combines --wait (wait for all resources Ready) with automatic rollback if timeout is exceeded or any resource fails. Standard production upgrade pattern.

Q24. How do you render a chart's templates locally without a cluster?

```
helm template [flags]
```

Example: `helm template my-release ./chart -f values.yaml > rendered.yaml`

No cluster connection needed. Use --debug for verbose output. Pipe to kubeconform for schema validation without a cluster.

Q25. How do you install a specific version of a chart?

```
helm install myapp bitnami/nginx --version 15.1.0
```

Find available versions: `helm search repo bitnami/nginx --versions` Always run `helm repo update` first to get the latest index.

Q26. How do you rollback to revision 2 of a release?

```
helm rollback my-nginx 2 -n production
```

Rollback to previous revision: `helm rollback my-nginx -n production` Rollback to last DEPLOYED (skips failed): `helm rollback my-nginx 0 -n production`

Q27. How do you uninstall a release but keep its history?

```
helm uninstall my-nginx -n production --keep-history
```

Without --keep-history all release Secrets are deleted and history is lost. With it, release status becomes 'uninstalled' and history is preserved for possible reinstall.

Q28. How do you see exactly what YAML Helm deployed for a running release?

```
helm get manifest my-nginx -n production
```

Retrieves the rendered YAML from the stored release Secret — exactly what was applied. May differ from current live state if resources were manually modified since deployment.

Q29. How do you see the diff before running helm upgrade?

```
helm diff upgrade myapp ./chart -f values.yaml -n production
```

Requires the helm-diff plugin. Shows added/removed/changed lines — like git diff but for Helm releases. Essential for change management approvals and preventing surprises.

Q30. How do you search for all available versions of a chart?

```
helm search repo bitnami/nginx --versions
```

Without `--versions` only the latest version is shown. Also: `helm search repo bitnami/nginx --version '~15.1.0'` for a range.

Q31. What does helm repo update do and when should you run it?

`helm repo update` fetches the latest `index.yaml` from all configured repositories and updates the local cache. Without it, `helm search repo` and `helm install` use stale data. Run before: any helm install/upgrade, in CI/CD pipelines, before `helm search repo`.

Q32. What is the difference between helm search hub and helm search repo?

`helm search repo`: queries the LOCAL CACHE of configured repos (must `helm repo add` first).

`helm search hub`: queries ARTIFACT HUB (artifacthub.io) over the internet — indexes charts from hundreds of public repos. No local configuration needed.

Q33. How do you pull a chart to inspect it locally without installing?

```
helm pull bitnami/nginx # downloads .tgz
helm pull bitnami/nginx --untar # extracts to ./nginx/
helm pull bitnami/nginx --version 15.1.0
```

Without downloading: `helm show values bitnami/nginx` (just `values.yaml`)

Q34. How do you pass a multi-line string as a helm value?

Use `--set-file` to read from a file: `helm install myapp ./chart --set-file config=./app.conf`

Or use a values file with YAML block scalar: `# override.yaml: config: |\n line1\n line2` `helm install myapp ./chart -f override.yaml`

Q35. How do you install the same chart twice in the same namespace?

Use different release names: `helm install app-blue ./chart --set color=blue -n prod` `helm install app-green ./chart --set color=green -n prod`

The chart MUST use `{{ include 'chart.fullname' . }}` for resource names — otherwise both installs create identically named resources and conflict.

Q36. What does helm install --generate-name do?

Generates a unique release name: `chart-name + random suffix` (e.g. `nginx-1699547823`). Customise: `--name-template 'myapp-{{randAlpha 6 | lower}}'`

Q37. How do you limit the number of release revisions stored?

Set `HELM_MAX_HISTORY=5` environment variable (default: 10). Or per-upgrade: `helm upgrade myapp ./chart --history-max 5`

Old revisions are pruned automatically when the limit is exceeded.

Q38. How do you target a different Kubernetes cluster?

```
helm install myapp ./chart --kube-context production
```

Or: KUBECONFIG=/path/to/prod.yaml helm install myapp ./chart Or set HELM_KUBECONTEXT environment variable for all commands.

Q39. What is the difference between helm uninstall with and without --keep-history?

Without --keep-history: removes all K8s resources AND all release Secrets (history lost permanently).

With --keep-history: removes K8s resources but keeps Secrets with status 'uninstalled'. Subsequent install with same name recognises existing history.

Q40. How do you check current Helm environment configuration?

```
helm env
```

Prints all HELM_* variables with current values — shows HELM_NAMESPACE, HELM_DRIVER, HELM_MAX_HISTORY, cache/config/data paths, etc.

Q41. What is helm package and what does it create?

```
helm package ./mychart
```

Creates mychart-.tgz where version comes from Chart.yaml. The .tgz is the distributable chart archive used for helm push, helm repo index, and direct install.

Q42. How does helm repo index work?

```
helm repo index ./releases/ --url https://charts.example.com
```

Generates index.yaml by scanning all .tgz files and recording their metadata. The index.yaml is what helm repo add points to. Run after adding new charts.

Q43. What is helm verify?

```
helm verify mychart-1.0.0.tgz
```

Checks: (1) .prov file exists alongside .tgz. (2) PGP signature in .prov is valid. (3) SHA-256 hash in .prov matches the actual .tgz. If any check fails, installation is aborted.

Q44. What are Helm starters?

Starters are template charts in \$HELM_DATA_HOME/starters/. When you run helm create --starter=, Helm copies the starter chart replacing the placeholder name with your chart name. Used to enforce organisational standards across all charts.

Q45. How do you run Helm tests?

```
helm test -n
```

Helm creates Pods/Jobs annotated with helm.sh/hook: test. Test succeeds if all pods exit with code 0. View logs: helm test --logs.

Q46. What does helm completion do?

```
helm completion bash > /etc/bash_completion.d/helm (or source <(helm completion bash))
```

Generates shell completion for bash, zsh, fish, and PowerShell. Critical for CKA/CKAD exam — enables Tab completion for release names, chart names, and flags.

Q47. How do you list releases in ALL namespaces?

```
helm list -A (or --all-namespaces)
```

Equivalent to `kubectl get pods -A` but for Helm releases. Shows NAME, NAMESPACE, REVISION, STATUS, CHART.

Q48. How do you upgrade with a completely clean values state?

```
helm upgrade myapp ./chart -f prod-values.yaml --reset-values
```

`--reset-values` discards ALL previously supplied values and starts fresh from chart defaults plus the new `-f` and `--set` flags. Most deterministic upgrade pattern.

Q49. What does helm get hooks show?

```
helm get hooks -n
```

Shows the rendered YAML of all hook resources (Jobs/Pods with `helm.sh/hook` annotations). Useful for debugging hook failures.

Q50. How do you add the bitnami repository and install PostgreSQL?

```
helm repo add bitnami https://charts.bitnami.com/bitnami helm repo update helm install my-pg bitnami/postgresql \
--set auth.postgresPassword=secret \
--namespace databases --create-namespace
```

Q51. What status values can appear in helm list?

deployed: current active revision, healthy. failed: last operation failed. pending-install/upgrade/rollback: in progress or stuck. superseded: replaced by a newer revision. uninstalled: removed with `--keep-history`.

Q52. What is the --cleanup-on-fail flag?

If an upgrade fails, `--cleanup-on-fail` deletes newly created resources that were not present before the upgrade attempt. Without it, partial upgrade resources remain in the cluster alongside the rolled-back state.

Q53. How do you force a resource to be recreated (delete + create) during upgrade?

```
helm upgrade myapp ./chart --force
```

Deletes and recreates resources rather than patching them. Use when a field that cannot be patched in-place needs to change. Causes brief downtime for affected resources.

Q54. How do you install a chart without waiting for resources to be Ready?

Simply do NOT use `--wait` or `--atomic`. By default, `helm install/upgrade` returns immediately after submitting manifests to the API server, without waiting for pods to become Running/Ready.

Q55. What is helm show all?

```
helm show all bitnami/nginx
```

Prints in sequence: Chart.yaml content, values.yaml content, and README.md. A quick way to inspect all chart documentation before installing.

CHAPTER 18

Section C: Templating Questions (Q56–Q70)

Q56. What is the difference between include and template?

template: writes output directly — cannot be piped. `{{- template 'mychart.labels' . }}` — cannot add | nindent after it.

include: returns text as a string that CAN be piped. `{{- include 'mychart.labels' . | nindent 4 }}`. ALWAYS use include.

Q57. Explain the difference between {{ }}, {{- }}, and {{- -}}.

`{{ }}`: no whitespace trimming — preserves all surrounding spaces and newlines. `{{- }}`: strips whitespace/newlines BEFORE the action. `{{- -}}`: strips whitespace/newlines AFTER the action. `{{- -}}`: strips both sides. Critical in YAML because extra blank lines affect parsing.

Q58. What does nindent do? Why is it preferred over indent?

indent N: adds N spaces to every line. No leading newline. nindent N: adds a newline FIRST, then N spaces. Equivalent to `\n + indent N`.

nindent is used because embedded multi-line values must start on a new line relative to the parent key. Without the newline, the first line appears on the same line as the key, producing invalid YAML.

Q59. Write a template that makes a value required with a custom error message.

```
image: "{{ .Values.image.repository }}:{{ required \"image.tag is required\" .Values.image.tag }}"
```

required returns the value if non-empty; if empty, template rendering fails with the provided message.

Q60. What are the .Release object fields?

.Release.Name: release name. .Release.Namespace: target namespace. .Release.IsInstall: true on first install. .Release.IsUpgrade: true on upgrade or rollback. .Release.Revision: integer, starts at 1. .Release.Service: always 'Helm'.

Q61. How do you use .Capabilities to choose the right API version?

```
{{- if .Capabilities.APIVersions.Has "networking.k8s.io/v1/Ingress" }}    apiVersion:
networking.k8s.io/v1 {{- else }} apiVersion: extensions/v1beta1 {{- end }}
```

Also useful: checking K8s version with semverCompare, checking if a CRD exists.

Q62. How do you iterate over a list from values.yaml?

```
values.yaml: extraEnvVars: [{name: LOG_LEVEL, value: info}]
```

```
template: env:\n{{- range .Values.extraEnvVars }}\n - name: {{ .name }}\n value: {{ .value | quote
}}\n{{- end }}
```

Inside range, . refers to the current list element.

Q63. How do you iterate over a map from values.yaml?

values.yaml: labels: {team: platform, env: production}

```
template: {{- range $key, $value := .Values.labels }}\n {{ $key }}: {{ $value | quote }}\n{{- end }}
```

Use \$key and \$value to unpack each map entry.

Q64. Why must you use \$ inside range/with to access the root context?

Inside range and with blocks, . (dot) changes scope to the current element/value. To access root-level objects like .Release.Name or .Values.global.*, you must use \$ which always refers to the root context. Example: {{ \$.Release.Name }} inside a range loop.

Q65. What is the tpl function?

tpl renders a string value as a Go template at runtime. Use case: allow users to put template expressions in values.yaml.

```
values.yaml: annotation: 'by={{ .Release.Name }}' template: annotation: {{ tpl .Values.annotation . }}
```

Danger: tpl evaluates arbitrary template code. Only use when you trust the values source.

Q66. How do you embed a values.yaml map into a Kubernetes resource?

Use toYaml | nindent:

```
resources: {{- toYaml .Values.resources | nindent 2 }}
```

toYaml serialises the Go map to YAML string; nindent 2 adds newline then 2-space indent.

Q67. How do you conditionally render an entire template file?

Wrap the entire file in an if guard:

```
{{- if .Values.ingress.enabled -}} apiVersion: networking.k8s.io/v1 ... {{- end }}
```

If .Values.ingress.enabled is false/nil, the file produces no output and no resource is created.

Q68. How do you define and use a named template?

```
In _helpers.tpl: {{- define "mychart.labels" -}}\napp.kubernetes.io/name: ...\n{{- end }}
```

```
In deployment.yaml: metadata:\n labels:\n {{- include "mychart.labels" . | nindent 4 }}
```

Q69. What does the default function do?

```
{{ .Values.image.tag | default "latest" }}
```

Returns the value if non-empty (non-zero, non-nil, non-empty string/list/map). If empty, returns the default. Used everywhere to provide fallbacks for optional values.

Q70. How does the with block simplify access to nested values?

with changes . to the specified value and executes the block only if non-empty:

```
{{- with .Values.resources }} resources:\n {{- toYaml . | nindent 2 }}\n{{- end }}
```

Avoids repeating the full path each time. Use \$ to access root context inside the block.

CHAPTER 19

Section D: Hooks, Security, Dependencies, CI/CD (Q71–Q95)

Q71. What are Helm hooks? List all hook types.

Hooks are K8s resources annotated with `helm.sh/hook` that Helm executes at lifecycle points. Types: pre-install, post-install, pre-upgrade, post-upgrade, pre-rollback, post-rollback, pre-delete, post-delete, and test.

Q72. How do you define a Job as a pre-upgrade hook?

Add to Job metadata: annotations: `{'helm.sh/hook': pre-upgrade, 'helm.sh/hook-weight': '-5', 'helm.sh/hook-delete-policy': before-hook-creation, hook-succeeded}`

The Job must have `restartPolicy`: Never or OnFailure.

Q73. What is hook-weight and how does it control order?

hook-weight is a string integer annotation. Hooks with LOWER (more negative) weights run FIRST. Default is 0. Value MUST be a string: '-5', not -5. Hooks with equal weight run in alphabetical order by resource name.

Q74. Explain each hook-delete-policy value.

`before-hook-creation` (default): delete before creating a new one — prevents accumulation.

`hook-succeeded`: delete after successful completion — keeps failed resources for debugging.

`hook-failed`: delete after failure — auto-cleans failed jobs.

Q75. What happens if a hook fails?

Helm marks the entire release as 'failed' and stops processing. NOT automatically rolled back unless `--atomic` was used. To recover: delete the failed hook resource manually, fix the issue, then retry the helm operation.

Q76. How do you write a Helm test?

Create a Pod in `templates/tests/` with: annotations: `{'helm.sh/hook': test, 'helm.sh/hook-delete-policy': before-hook-creation, hook-succeeded}`

The pod runs a validation command. Exit code 0 = test pass. Run with: `helm test -n`

Q77. How does Helm store release secrets? What encoding is used?

Kubernetes Secret type `helm.sh/release.v1`, one per revision. Data field 'release' is: JSON payload → gzip → base64 → base64 (outer K8s Secret encoding). Decode: `base64 -d | base64 -d | gunzip | python3 -m json.tool`

Q78. Why is --set password=secret insecure?

(1) Shell history (~/.bash_history) records the full command. (2) Process list (ps aux) shows running command arguments. (3) The value is stored in plain base64 in the release Secret — readable by anyone with 'get secrets' permission. (4) CI/CD logs may capture the full command.

Q79. What is the helm-secrets plugin?

helm-secrets integrates Mozilla SOPS for transparent encryption/decryption of values files. Workflow: helm secrets enc secrets.yaml (encrypts with KMS/PGP), commit encrypted file to git, helm secrets install myapp ./chart -f secrets.yaml (decrypts at deploy time). SOPS supports AWS KMS, GCP KMS, Azure Key Vault, PGP.

Q80. What RBAC permissions does a service account need for helm install?

Secrets: get/list/create/update/delete (for release storage). Plus all resource types the chart creates: Deployments, Services, ConfigMaps, ServiceAccounts, Ingresses, etc. For cluster-scoped resources (ClusterRole, CRDs): ClusterRole permissions needed.

Q81. How do you sign and verify a Helm chart?

Sign: helm package ./chart --sign --key 'My Key' --keyring ~/.gnupg/secring.gpg Creates: chart-1.0.0.tgz + chart-1.0.0.tgz.prov

Verify: helm verify chart-1.0.0.tgz --keyring ~/.gnupg/pubring.gpg Install verified: helm install myapp chart-1.0.0.tgz --verify

Q82. What is a provenance (.prov) file?

The .prov file contains: (1) Chart metadata from Chart.yaml. (2) SHA-256 hash of the .tgz archive. (3) PGP signature of the hash using the signer's private key. Recipients verify with the signer's public key to confirm the chart was not tampered with.

Q83. How do you add a chart dependency and download it?

In Chart.yaml: dependencies: [{name: postgresql, version: '12.x.x', repository: 'https://charts.bitnami.com/bitnami', condition: postgresql.enabled}]

Then: helm dependency update (downloads, creates Chart.lock) CI: helm dependency build (uses Chart.lock exactly, deterministic)

Q84. How do you disable a sub-chart dependency?

In Chart.yaml: condition: postgresql.enabled In values.yaml: postgresql:\n enabled: false

Or at install: helm install myapp ./chart --set postgresql.enabled=false

When disabled, sub-chart templates are not rendered and no resources are created.

Q85. What are global values?

A 'global' key in values.yaml is accessible to ALL sub-charts as .Values.global.*. Example: global: {imageRegistry: 'harbor.example.com'}

Sub-chart template: {{ .Values.global.imageRegistry }}/{{ .Values.image.repository }}

Used for: registry overrides, pull secrets, storage class, environment name.

Q86. Describe a production-safe helm upgrade pipeline.

1. helm lint --strict. 2. helm template > manifest.yaml. 3. kubeconform (schema validate). 4. conftest (OPA policies). 5. helm diff upgrade (preview changes). 6. Manual approval gate. 7. helm upgrade --install --atomic --timeout 10m. 8. helm test.

Q87. What is Helmfile?

Helmfile is a declarative configuration file for managing multiple Helm releases. Benefits over multiple helm install commands: (1) Idempotent — helmfile apply installs missing, upgrades changed. (2) Built-in environment support. (3) 'needs' field for dependency ordering. (4) Single command for diff/sync/destroy.

Q88. How does ArgoCD use Helm? Does it run helm install?

ArgoCD does NOT run helm install. It runs helm template and applies with kubectl apply. Consequences: hooks do NOT run, no helm history, NOTES.txt not shown.

Flux HelmRelease CRD uses the Helm SDK directly — native install/upgrade, so hooks work and history is maintained.

Q89. What is the helm-diff plugin?

helm diff upgrade shows the diff between deployed state and proposed upgrade — like git diff but for Helm. Essential for: previewing changes before production deploys, detecting unintended regressions, and satisfying change management approval requirements.

Q90. You get 'another operation is in progress'. What caused it and how do you fix it?

Cause: a previous upgrade was interrupted (timeout, Ctrl+C), leaving the release in pending-upgrade state. Helm uses release status as a lock.

Fix: helm rollback myapp 0 -n production (resets to last deployed revision)

Nuclear: delete all release secrets for the release, then reinstall.

Q91. Helm status shows 'deployed' but a Deployment was manually deleted. How do you reconcile?

Helm doesn't continuously monitor cluster state — its status reflects the last operation. Reconcile by running helm upgrade (which re-applies all manifests). Helm's 3-way merge detects the missing Deployment and recreates it.

For continuous reconciliation, use ArgoCD or Flux instead of raw Helm.

Q92. All revisions are in 'failed' status. How do you recover?

helm history myapp — confirm all revisions are failed.

Try: helm rollback myapp

If all fail: helm uninstall myapp (force cleanup), then helm install fresh. Restore PVC data from backup if stateful.

Q93. A template fails with 'nil pointer evaluating interface {}.tag'. How do you fix it?

The parent key is nil (e.g. .Values.image is missing from values.yaml). Fixes: 1. Ensure values.yaml has: image:\n tag: 'alpine' 2. Use 'with' guard: {{- with .Values.image }}...{{- end }} 3. Use default: {{ .Values.image.tag | default 'latest' }}

Q94. Your chart works in dev but fails in prod because prod lacks a CRD.

```
{{- if .Capabilities.APIVersions.Has "cert-manager.io/v1/Certificate" }} apiVersion: cert-manager.io/v1
kind: Certificate ... {{- end }}
```

Also add a values.yaml toggle (certManager.enabled: true) and document the requirement in NOTES.txt.

Q95. How do you enforce resource limits on all charts in production?

Three layers: (1) values.schema.json: require resources.limits to be present. (2) Conftest/OPA in CI: helm template | conftest test with a Rego policy that denies containers missing resource limits. (3) Kyverno or OPA Gatekeeper admission webhooks: block Deployments without limits at the cluster level regardless of source.

PART 7

PART 7: QUICK REFERENCE

CHAPTER 20

Helm Cheat Sheet

All Commands at a Glance

Command	Description
<code>helm install</code>	Install a release
<code>helm install --atomic</code>	Install with wait + auto-rollback on failure
<code>helm install --dry-run</code>	Simulate — hits API server, no deploy
<code>helm install --generate-name</code>	Auto-generate release name
<code>helm install -f v.yaml</code>	Install with custom values file
<code>helm install --set k=v</code>	Install with inline override (highest priority)
<code>helm upgrade</code>	Upgrade a release
<code>helm upgrade --install</code>	Upgrade or install if not exists (idempotent)
<code>helm upgrade --atomic</code>	Upgrade with wait + auto-rollback
<code>helm upgrade --reset-values</code>	Discard all previous values
<code>helm upgrade --reuse-values</code>	Keep previous values (avoid in CI)
<code>helm list</code>	List releases in current namespace
<code>helm list -A</code>	List releases in ALL namespaces
<code>helm list --all</code>	Include failed/pending releases
<code>helm status</code>	Show status summary + NOTES.txt
<code>helm get manifest</code>	Show deployed K8s manifests exactly as applied
<code>helm get values</code>	Show user-supplied values only
<code>helm get values --all</code>	Show ALL values (defaults + user)
<code>helm get hooks</code>	Show hook manifests
<code>helm get notes</code>	Show NOTES.txt output
<code>helm history</code>	Show full revision history
<code>helm rollback</code>	Rollback to specific revision
<code>helm rollback 0</code>	Rollback to last deployed (skips failed)
<code>helm uninstall</code>	Uninstall — removes all resources
<code>helm uninstall --keep-history</code>	Uninstall but preserve history

Command	Description
helm template	Render templates locally (no cluster)
helm lint	Check chart for errors
helm lint --strict	Fail on warnings too
helm test	Run test hooks in cluster
helm repo add	Add a repository
helm repo list	List configured repositories
helm repo update	Fetch latest index.yaml from all repos
helm repo remove	Remove a repository
helm search repo	Search local repo cache
helm search hub	Search Artifact Hub (internet)
helm show chart	Show Chart.yaml metadata
helm show values	Show default values.yaml
helm show all	Show all chart info (chart + values + readme)
helm dependency update	Download deps, regenerate Chart.lock
helm dependency build	Download deps from Chart.lock exactly (CI)
helm dependency list	List resolved dependencies
helm package	Create .tgz chart archive
helm package --sign --key 'K'	Create signed chart with .prov provenance file
helm verify	Verify GPG signature and SHA-256 hash
helm repo index	Generate repository index.yaml
helm registry login	Authenticate to OCI registry
helm push oci://url	Push chart to OCI registry
helm pull oci://url --version v	Pull chart from OCI registry
helm version	Show Helm version
helm env	Show all HELM_* environment variables
helm plugin install	Install a plugin from Git URL
helm plugin list	List installed plugins
helm completion bash	Generate bash completion script

HELM_* Environment Variables

Variable	Description
HELM_CACHE_HOME	Local chart cache (repos & downloaded charts)
HELM_CONFIG_HOME	Config directory (repositories.yaml, registry auth)
HELM_DATA_HOME	Data directory (plugins, starters)
HELM_DEBUG	Set 1 for verbose debug output
HELM_DRIVER	Release storage: secret (default) configmap memory sql
HELM_KUBEAPISERVER	Override Kubernetes API server URL
HELM_KUBECONTEXT	Override kubeconfig context
HELM_KUBETOKEN	Bearer token for K8s API auth (CI/CD service accounts)
HELM_KUBEINSECURE_SKIP_TLS_VERIFY	Skip TLS verification — dev only!
HELM_NAMESPACE	Default namespace for all commands
HELM_MAX_HISTORY	Max revision history per release (default: 10)
HELM_PLUGINS	Override plugins directory path
HELM_REGISTRY_CONFIG	OCI registry auth config path
HELM_REPOSITORY_CACHE	Chart repository cache path
HELM_REPOSITORY_CONFIG	Override repositories.yaml path

Built-in Template Objects

Object.Field	Value / Notes
.Release.Name	Release name (e.g. my-nginx)
.Release.Namespace	Target namespace
.Release.IsInstall	bool — true on initial install
.Release.IsUpgrade	bool — true on upgrade / rollback
.Release.Revision	int — starts at 1, increments each operation
.Release.Service	Always 'Helm'
.Chart.Name	Chart name from Chart.yaml
.Chart.Version	Chart version (SemVer 2)
.Chart.AppVersion	App version — informational only, Helm ignores it

Object.Field	Value / Notes
.Values.*	Merged values — user overrides + chart defaults
.Capabilities.KubeVersion.GitVersion	Full K8s version string e.g. v1.28.3
.Capabilities.APIVersions.Has 'a/v/K'	bool — check API group/version/kind availability
.Files.Get 'path'	Read a non-template file from the chart directory
.Template.Name	Current template file path (for error messages)

Hook Types Reference

Hook Type	Runs When	Recommended Delete Policy
pre-install	Before chart resources created	before-hook-creation, hook-succeeded
post-install	After all resources Ready	before-hook-creation, hook-succeeded
pre-upgrade	Before upgrade applied	before-hook-creation, hook-succeeded
post-upgrade	After upgrade complete	before-hook-creation, hook-succeeded
pre-delete	Before uninstall begins	before-hook-creation
post-delete	After all resources deleted	before-hook-creation
pre-rollback	Before rollback begins	before-hook-creation
post-rollback	After rollback complete	before-hook-creation
test	Only when helm test is run	before-hook-creation, hook-succeeded

Common Error Messages and Fixes

Error Message	Cause	Fix
cannot re-use a name still in use	Release name exists	helm upgrade --install or choose new name
another operation is in progress	Previous op left pending state	helm rollback 0
release has no deployed releases	All revisions failed	helm uninstall + reinstall
nil pointer evaluating interface {}.tag	Parent key nil in values	Add to values.yaml; use 'with' guard
value is required	required function triggered	Supply via --set or -f
INSTALLATION FAILED: timed out	--atomic timeout exceeded	Increase --timeout; debug pod scheduling
rendered manifests contain existing resource	Resource exists outside Helm	helm upgrade --force; or delete orphan

Error Message	Cause	Fix
no repositories configured	helm repo add not run	helm repo add ; helm repo update
cannot unmarshal !!str into int32	Wrong YAML type in values	Use --set-string or quote value in YAML

Values Precedence (Lowest → Highest)

1. Chart defaults — values.yaml bundled with chart
2. Sub-chart values.yaml defaults
3. Parent chart overrides for sub-chart keys
4. User -f values files — multiple files: left-to-right, later wins
5. --set flags — always override everything

EXAM TIP: Final exam tip: The three most-tested Helm scenarios in CKA / CKAD / KCNA are: (1) helm upgrade --install with --atomic and custom values files, (2) helm rollback to a specific revision, and (3) reading helm history output correctly. Practice these until they are muscle memory.